Leibniz Universität Hannover

L3S Research Center

Knowledge-Based Systems Group

L3S-Offshore-2 Frontend

Web-based Petri Net Visualization and Simulation Tool

Technical Documentation

Version 1.0

Author

Martin Krauser
martin.krauser@stud.uni-hannover.de

Supervisor

Peng Qian

Project Period: March 2025 – January 2026

Hannover, November 2025

Document Information

Project: L3S-Offshore-2 Frontend
Version: 1.0
Date: November 2025
Author: Martin Krauser
Institution: L3S Research Center

Version History

Version	Date	Author	Changes
1.0	November 2025	Martin Krauser	Initial version

License

Abstract

This document provides comprehensive technical documentation for the L3S-Offshore-2 Frontend, a web-based application for Petri net visualization, manipulation, and simulation. The frontend was developed as part of the L3S-Offshore-2 research project at the L3S Research Center, Leibniz Universität Hannover.

The application is built using Angular and provides the following core capabilities:

- **Visualization:** Interactive rendering of Petri nets from PNML files with automatic layout algorithms (Sugiyama)
- **Manipulation:** Drag-and-drop editing of places, transitions, and arcs with real-time token modification
- **Simulation:** Step-by-step and automated playback of firing sequences with transition highlighting
- **Analysis:** Display of place invariants and transition firing statistics from multi-run simulations
- **Offshore Scheduling:** Dedicated parameter interface for offshore wind farm installation scheduling models

The documentation covers the system architecture, key implementation details, deployment procedures, and provides guidance for future development and maintenance.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Project Context

The L3S-Offshore-2 project is a research initiative at the L3S Research Center focusing on the application of Petri nets for modeling and simulating offshore wind farm installation logistics. Efficient scheduling of vessel operations, component installations, and resource management is critical for minimizing costs and maximizing the utilization of weather windows.

This documentation describes the development of a web-based frontend application that enables researchers and practitioners to:

- Visualize complex Petri net models in an interactive browser environment
- Modify and extend existing models through a graphical user interface
- Execute and analyze simulations to evaluate scheduling strategies
- Configure domain-specific parameters for offshore logistics scenarios

The frontend integrates with a Python-based backend (Flask) that provides simulation capabilities using the `gspn4py` library for Generalized Stochastic Petri Nets.

1.2 Problem Statement

Prior to this project, the research group relied on disconnected tools for Petri net modeling:

1. **Visualization:** Desktop applications (e.g., WoPeD) that could not be easily shared
2. **Simulation:** Python scripts executed via command line

3. **Parameter Configuration:** Manual editing of configuration files

This fragmentation created barriers for:

- Collaboration between researchers
- Rapid prototyping and iteration on models
- Demonstration of results to external stakeholders

1.3 Solution Approach

The solution is a single-page web application (SPA) that consolidates all workflows into an integrated environment. The key design decisions include:

- **Angular Framework:** Chosen for its component-based architecture and TypeScript support, enabling maintainable and type-safe code
- **Fork of Existing Project:** Instead of building from scratch, the project forked the open-source "PNML-Frontend" by i-love-petrinets, which provided a solid foundation for PNML parsing and basic visualization
- **Tab-Based Navigation:** Clear separation of concerns (Build, Code, Simulation, Analyze, Save, Offshore) for different user workflows
- **Containerized Deployment:** Docker-based deployment for easy installation on research servers

1.4 Contributions

The main contributions of this project are:

1. **Extended Visualization:**
   - Viewport-centered zooming and panning with ResizeObserver integration
   - Automatic layout via Sugiyama algorithm for PNML files without coordinates
   - Tab-aware UI state management
2. **Simulation Capabilities:**
   - Animated playback of firing sequences from backend simulation
   - Manual "Token Game" mode for step-by-step execution
   - Multi-run simulation with transition firing frequency statistics

3. **Offshore-Specific Integration:**
    - Dedicated parameter input panel for offshore scheduling models
    - Integration with backend scheduling endpoints
4. **Production-Ready Deployment:**
    - Docker containerization with nginx reverse proxy
    - Deployment on L3S research infrastructure

1.5 Document Structure

This documentation is organized as follows:

Chapter 2: Provides background on Petri nets, the PNML format, and relevant web technologies (Angular, TypeScript).

Chapter 3: Describes the overall system architecture, component hierarchy, and service dependencies.

Chapter 4: Details key implementation aspects including the zoom system, simulation logic, and tab-state management.

Chapter 5: Covers the Docker-based deployment process and production configuration.

Chapter 6: Discusses related tools and how this project compares.

Chapter 7: Summarizes achievements and outlines future work.

Appendix A: Contains supplementary material such as API documentation and configuration files.

Chapter 2

Fundamentals

This chapter provides the theoretical and technical background necessary to understand the implementation details described in later chapters. It covers the fundamentals of Petri nets, the PNML exchange format, and the web technologies used in the frontend development.

2.1 Petri Nets

Petri nets are a mathematical modeling language for the description of distributed systems. Originally introduced by Carl Adam Petri in his 1962 dissertation, they provide a graphical and formal representation of concurrent processes.

2.1.1 Basic Definitions

A Petri net is a directed bipartite graph consisting of:

- **Places** (P): Represented as circles, places hold tokens and model conditions or states
- **Transitions** (T): Represented as rectangles or bars, transitions model events or actions
- **Arcs** (F): Directed edges connecting places to transitions (input arcs) or transitions to places (output arcs)
- **Marking** (M): The distribution of tokens across places, representing the system state

Formally, a Petri net is defined as a tuple $N = (P, T, F, W, M_0)$ where:

- P is a finite set of places
- T is a finite set of transitions, $P \cap T = \emptyset$

- $F \subseteq (P \times T) \cup (T \times P)$ is the flow relation
- $W : F \to \mathbb{N}^+$ is the arc weight function
- $M_0 : P \to \mathbb{N}$ is the initial marking

2.1.2 Firing Rules

A transition $t \in T$ is **enabled** at marking M if and only if:

$$\forall p \in {\bullet t} : M(p) \geq W(p, t)$$

where ${\bullet t}$ denotes the preset of t (input places). When an enabled transition fires, the marking changes according to:

$$M'(p) = M(p) - W(p, t) + W(t, p)$$

2.1.3 Extensions: Stochastic Petri Nets

For modeling real-world systems with timing, Generalized Stochastic Petri Nets (GSPNs) extend basic Petri nets with:

- **Timed transitions**: Fire after a random delay drawn from an exponential distribution
- **Immediate transitions**: Fire instantaneously when enabled (priority over timed transitions)

The L3S-Offshore-2 project uses GSPNs to model the stochastic nature of offshore installation operations, including weather-dependent delays.

2.2 PNML: Petri Net Markup Language

PNML (Petri Net Markup Language) is an XML-based interchange format for Petri nets, standardized as ISO/IEC 15909-2. It provides a common syntax for storing and exchanging Petri net models across different tools.

2.2.1 Document Structure

A PNML document has the following hierarchical structure:

```
<?xml version="1.0" encoding="UTF-8"?>
<pnml>
  <net id="net1" type="http://www.pnml.org/version-2009/grammar
    /ptnet">
    <page id="page1">
      <place id="p1">
        <name><text>Place 1</text></name>
        <initialMarking><text>1</text></initialMarking>
```

```
        <graphics><position x="100" y="100"/></graphics>
      </place>
      <transition id="t1">
        <name><text>Transition 1</text></name>
        <graphics><position x="200" y="100"/></graphics>
      </transition>
      <arc id="a1" source="p1" target="t1">
        <inscription><text>1</text></inscription>
      </arc>
    </page>
  </net>
</pnml>
```

Listing 2.1: Basic PNML structure

2.2.2 Graphical Information

PNML files may optionally contain graphical information (positions, dimensions) in `<graphics>` elements. When this information is missing or invalid, the frontend applies the Sugiyama algorithm to generate a readable layout automatically.

2.3 Web Technologies

The frontend is built using modern web technologies that enable rich, interactive single-page applications.

2.3.1 Angular Framework

Angular is a TypeScript-based web application framework developed by Google. Key concepts used in this project include:

- **Components**: Reusable UI building blocks with encapsulated logic, templates, and styles
- **Services**: Singleton classes for shared state and business logic, injected via dependency injection
- **Observables (RxJS)**: Reactive programming patterns for handling asynchronous events and data streams
- **Modules**: Organizational units that group related components, services, and other code

2.3.2 SVG for Petri Net Rendering

Scalable Vector Graphics (SVG) is used for rendering Petri nets because:

- Resolution-independent rendering at any zoom level
- DOM integration allows event handling on individual elements
- CSS styling and animations can be applied
- Native browser support without plugins

2.3.3 Angular Material

Angular Material provides pre-built UI components following Google's Material Design guidelines. The project uses Material components for:

- Tab navigation (`mat-tab-group`)
- Buttons and form controls
- Dialogs for warnings and information
- Icons and tooltips

2.4 Related Concepts

Chapter 3

System Architecture

This chapter describes the overall architecture of the L3S-Offshore-2 Frontend, including its integration with the backend, the component hierarchy, and the service layer design.

3.1 System Overview

The L3S-Offshore-2 system follows a client-server architecture with clear separation between the visualization frontend and the simulation backend.

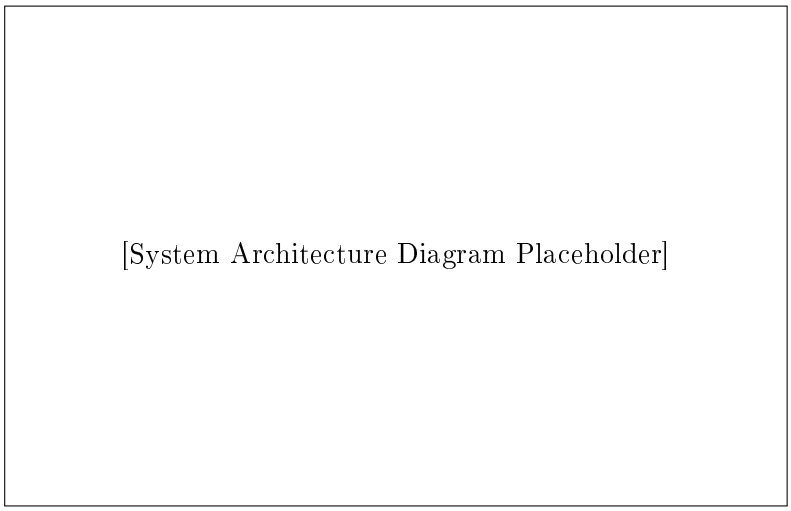

Figure 3.1: High-level system architecture showing frontend-backend interaction

3.1.1 Frontend (Angular)

The frontend is a single-page application (SPA) responsible for:

- Rendering Petri nets as interactive SVG graphics
- Handling user interactions (editing, navigation, simulation control)
- Managing application state across different tabs
- Communicating with the backend via REST API

3.1.2 Backend (Flask)

The Python backend provides:

- PNML parsing and validation
- Petri net simulation using the `gspn4py` library
- Offshore scheduling algorithms
- Example model storage and retrieval

3.2 Frontend Component Hierarchy

The Angular application is organized into a hierarchical component structure that reflects the UI layout.

3.2.1 Root Components

AppComponent The root component that bootstraps the application and contains the main layout

HeaderComponent Displays the project banner with L3S branding

FooterComponent Shows institutional logos with links to L3S and LUH homepages

3.2.2 Tab Components

The main content area uses Angular Material's tab group:

ButtonBarComponent The central coordinator that manages tab switching and toolbar rendering for each tab (Build, Code, Simulation, Analyze, Save, Offshore)

PetriNetComponent Renders the interactive SVG visualization of the Petri net

CodeEditorComponent Provides a text editor for direct PNML manipulation

ParameterTableComponent Displays and edits simulation parameters

OffshoreViewComponent Specialized view for offshore scheduling parameters

3.2.3 Dialog Components

Modal dialogs for user feedback and interaction:

DataPersistenceInfoDialogComponent Warns users about data loss on page reload

ErrorPopupComponent Displays error messages from backend or parsing failures

HelpPopupComponent Provides contextual help

TransitionFiringInfoPopupComponent Shows firing statistics for transitions

3.3 Service Layer

Services are singleton classes that provide shared functionality and state management.

3.3.1 Core Services

DataService Central state container for the Petri net model. Manages places, transitions, arcs, and the current marking. Emits change events via RxJS observables.

UiService Manages UI state including the current tab, simulation mode (automatic/manual), selected elements, and zoom level.

ZoomService Handles viewport transformations (pan, zoom) with ResizeObserver integration for responsive behavior.

ParserService Converts between PNML XML, internal data structures, and the JSON representation used by the backend.

3.3.2 Simulation Services

SimulationService Communicates with the backend's simulation endpoints. Manages firing sequences and multi-run results.

TokenGameService Implements the manual simulation mode where users can step through the firing sequence or click on enabled transitions.

PlanningService Handles the offshore scheduling API calls and parameter management.

3.3.3 Utility Services

ExportImageService Exports the current view as PNG

ExportSvgService Exports the Petri net as SVG

ExportJsonDataService Exports the internal representation as JSON

LayoutSugiyamaService Implements the Sugiyama algorithm for automatic layout

3.4 Data Flow

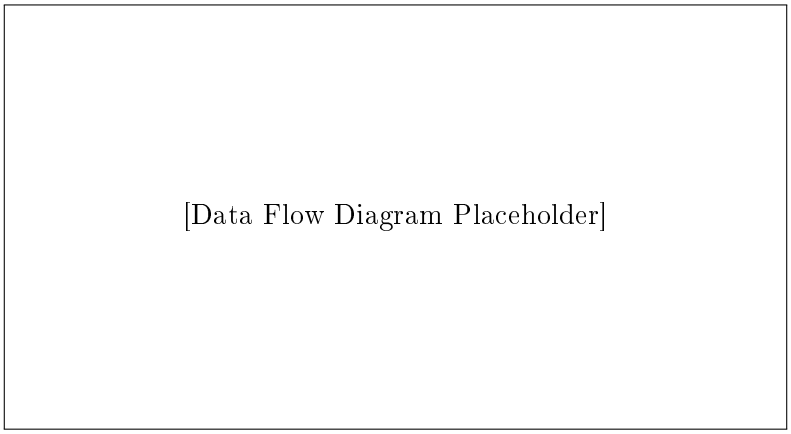

Figure 3.2: Data flow between components and services

The application follows a unidirectional data flow pattern:

1. **User Input:** User interacts with UI (clicks, drags, edits)
2. **Component Handler:** Component method processes the event
3. **Service Update:** Component calls service method to update state
4. **Observable Emission:** Service emits new state via RxJS Subject
5. **Subscriber Notification:** All subscribed components receive update
6. **View Re-render:** Angular change detection updates the DOM

3.5 Tab State Management

A key architectural feature is the tab-aware state management that ensures UI consistency when switching between tabs.

3.5.1 The Tab*Observable*

The `UiService` exposes a `tab$` BehaviorSubject that emits the current tab name whenever the user navigates:

```
private _tab$ = new BehaviorSubject<string>('build');

get tab$(): Observable<string> {
  return this._tab$.asObservable();
}

set tab(value: string) {
  console.log(`[UiService] Tab changed: ${this._tab$.value} ->
    ${value}`);
  this._tab$.next(value);
}
```

Listing 3.1: Tab state observable in UiService

3.5.2 Tab Guards in Components

Components subscribe to `tab$` and conditionally enable/disable features:

```
this.uiService.tab$.subscribe(tab => {
  if (tab !== 'simulation') {
    this.clearAllHighlights();
    this.stopAnimationTimer();
  } else {
    this.restoreSimulationState();
  }
});
```

Listing 3.2: Tab-aware highlighting in PetriNetComponent

Chapter 4

Implementation Details

This chapter describes the key implementation aspects of the frontend, focusing on the most complex and interesting technical solutions.

4.1 Zoom and Pan System

One of the most technically challenging aspects was implementing a robust zoom and pan system that behaves intuitively across different viewport sizes.

4.1.1 The Problem

The original fork had a basic zoom implementation that exhibited several issues:

- Content would "drift" away from the center when zooming
- Mouse coordinates were incorrect after panning
- Elements would not be placed at the click position in the Build tab
- The "Fit Content" feature would not trigger correctly after loading a file

4.1.2 Viewport-Centered Zooming

The solution involved centering all zoom operations around the viewport center rather than the origin:

```
zoomAtCenter(factor: number): void {
  const viewportCenterX = this.viewportWidth / 2;
  const viewportCenterY = this.viewportHeight / 2;

  // Transform viewport center to SVG coordinates
  const svgX = (viewportCenterX - this.panX) / this.scale;
  const svgY = (viewportCenterY - this.panY) / this.scale;
```

```
8
9   // Apply new scale
10  this.scale *= factor;
11  this.scale = Math.max(0.1, Math.min(5, this.scale));
12
13  // Adjust pan to keep the same SVG point at viewport center
14  this.panX = viewportCenterX - svgX * this.scale;
15  this.panY = viewportCenterY - svgY * this.scale;
16 }
```

Listing 4.1: Viewport-centered zoom calculation

4.1.3 ResizeObserver Integration

To handle viewport size changes (e.g., when switching from the Code tab's 50% width back to full width), a `ResizeObserver` was integrated:

```
1  private resizeObserver: ResizeObserver;
2  private shouldAutoFit = false;
3
4  ngAfterViewInit(): void {
5    this.resizeObserver = new ResizeObserver(entries => {
6      for (const entry of entries) {
7        this.zoomService.updateViewportDimensions(
8          entry.contentRect.width,
9          entry.contentRect.height
10       );
11       if (this.shouldAutoFit) {
12         this.fitContentToViewport();
13       }
14     }
15   });
16   this.resizeObserver.observe(this.svgContainer.nativeElement);
17 }
```

Listing 4.2: ResizeObserver setup for responsive viewport

4.2 Simulation Animation

The simulation animation system displays the firing sequence received from the backend.

4.2.1 Firing Sequence Format

The backend returns a firing sequence as an array of transition IDs:

```
1  interface SimulationResult {
2    firing_seq: string[];      // e.g., ["t1", "t3", "t2", "t1"]
3    detailed_log: StepLog[];   // Optional: token counts at each
       step
4  }
```

Listing 4.3: Firing sequence data structure

4.2.2 Animation Loop

The animation uses `setInterval` with a configurable speed:

```
private animationTimer: any;

startAutoplay(): void {
  const interval = 1000 / this.playbackSpeed;
  this.animationTimer = setInterval(() => {
    if (this.currentStep < this.firingSequence.length) {
      this.highlightTransition(this.firingSequence[this.
    currentStep]);
      this.currentStep++;
      this.timelinePosition = this.currentStep;
    } else {
      this.stopAutoplay();
    }
  }, interval);
}
```

Listing 4.4: Animation timer implementation

4.2.3 Transition Highlighting

Transitions are highlighted by applying CSS classes that trigger SVG animations:

```
highlightTransition(transitionId: string): void {
  // Remove previous highlight
  this.clearHighlights();

  // Find the SVG element
  const element = document.getElementById(`transition-${
    transitionId}`);
  if (element) {
    element.classList.add('firing');
    // CSS handles the animation:
    // .firing { fill: #ff6600; filter: drop-shadow(0 0 8px #
    ff6600); }
  }
}
```

Listing 4.5: Transition highlighting with CSS

4.3 Token Reset Logic

A critical feature is resetting tokens to the initial marking when leaving the Simulation tab.

4.3.1 Baseline Snapshot

When the user first enters the Simulation tab, a snapshot of the current marking is saved:

```
private baselineMarking: Map<string, number> = new Map();

saveBaselineMarking(): void {
  this.dataService.places.forEach(place => {
    this.baselineMarking.set(place.id, place.tokens);
  });
}

resetTokensToBaseline(): void {
  this.baselineMarking.forEach((tokens, placeId) => {
    const place = this.dataService.getPlaceById(placeId);
    if (place) {
      place.tokens = tokens;
    }
  });
  this.dataService.triggerDataChanged();
}
```

Listing 4.6: Baseline marking snapshot

4.3.2 Tab-Aware Reset

The reset is triggered when navigating away from the Simulation tab:

```
this.uiService.tab$.subscribe(newTab => {
  if (this.previousTab === 'simulation' && newTab !== '
    simulation') {
    this.resetTokensToBaseline();
    this.clearAllHighlights();
  }
  if (newTab === 'simulation' && this.previousTab !== '
    simulation') {
    this.saveBaselineMarking();
  }
  this.previousTab = newTab;
});
```

Listing 4.7: Tab-aware token reset

4.4 Parameter Table

The parameter table allows users to configure simulation parameters before running.

4.4.1 Dynamic Form Generation

Parameters are dynamically generated from the backend's DTO definition:

```
generateFormControls(parameters: ParameterDefinition[]): void {
  parameters.forEach(param => {
    this.formGroup.addControl(
      param.name,
      new FormControl(param.defaultValue, this.getValidators(
    param))
    );
  });
}
```

Listing 4.8: Dynamic parameter form generation

4.4.2 Reactive Forms Integration

The initial implementation caused severe UI freezes due to incorrect parent-child form integration. The solution used Angular's `ControlContainer`:

```
@Component({
  selector: 'app-parameter-row',
  viewProviders: [
    { provide: ControlContainer, useExisting:
    FormGroupDirective }
  ]
})
export class ParameterRowComponent {
  // Child component now correctly integrates with parent form
}
```

Listing 4.9: ControlContainer fix for nested forms

4.5 Right-Click Statistics

To avoid conflicts with the manual simulation mode (left-click fires transition), statistics are displayed via right-click.

```
@HostListener('contextmenu', ['$event'])
onRightClick(event: MouseEvent): void {
  if (this.uiService.tab !== 'simulation') return;

  event.preventDefault(); // Suppress browser context menu

  const transitionId = this.getTransitionIdFromEvent(event);
  if (transitionId) {
    const stats = this.simulationService.getTransitionStats(
    transitionId);
    this.showStatisticsPopup(event.clientX, event.clientY,
    stats);
  }
```

```
}
```

Listing 4.10: Right-click handler for statistics popup

4.6 Data Persistence Warning

Users are warned about data loss through two mechanisms:

4.6.1 Initial Dialog

A dialog appears on first visit (controlled by `localStorage`):

```
ngOnInit(): void {
  const shown = localStorage.getItem('dataPersistenceInfoShown');
  if (!shown) {
    this.dialog.open(DataPersistenceInfoDialogComponent);
  }
}
```

Listing 4.11: Data persistence dialog on app start

4.6.2 Beforeunload Guard

The browser's native confirmation dialog is triggered when closing with unsaved data:

```
@HostListener('window:beforeunload', ['$event'])
onBeforeUnload(event: BeforeUnloadEvent): void {
  if (!this.dataService.isEmpty()) {
    event.preventDefault();
    event.returnValue = ''; // Required for Chrome
  }
}
```

Listing 4.12: Beforeunload event handler

Chapter 5

Deployment

This chapter describes how to build and deploy the L3S-Offshore-2 Frontend, both locally for development and on production servers.

5.1 Prerequisites

5.1.1 Development Environment

The following software is required for local development:

- **Node.js:** Version 18.x or higher (LTS recommended)
- **npm:** Version 9.x or higher (comes with Node.js)
- **Angular CLI:** Version 17.x (`npm install -g @angular/cli`)
- **Git:** For version control

5.1.2 Production Environment

For production deployment:

- **Docker:** Version 20.x or higher
- **Docker Compose:** (optional, for multi-container setups)
- **nginx:** Included in the Docker image

5.2 Local Development

5.2.1 Clone and Install

```
# Clone the repository
git clone https://github.com/krausality/PNML_Frontend.git
cd PNML_Frontend

# Install dependencies
npm install
```

Listing 5.1: Cloning and installing dependencies

5.2.2 Development Server

Start the Angular development server with hot reloading:

```
# Start development server
ng serve

# Or with explicit host binding (for network access)
ng serve --host 0.0.0.0 --port 4200
```

Listing 5.2: Starting the development server

The application will be available at http://localhost:4200.

5.2.3 Backend Connection

The development environment expects the Flask backend to run on port 9040. Configure the backend URL in `src/environments/environment.ts`:

```
export const environment = {
  production: false,
  apiUrl: 'http://localhost:9040'
};
```

Listing 5.3: Development environment configuration

5.3 Production Build

5.3.1 Angular Production Build

Create an optimized production build:

```
# Production build with optimizations
ng build --configuration production

# Output is generated in dist/pnml-frontend/
```

Listing 5.4: Creating a production build

The production build includes:

- Ahead-of-Time (AOT) compilation

- Tree shaking to remove unused code
- Minification and uglification
- Content hashing for cache busting

5.3.2 Build Verification

Verify the build output:

```
# Check bundle sizes
ls -la dist/pnml-frontend/browser/

# Expected output: main.js, polyfills.js, styles.css
# Total size should be under 500KB gzipped
```

Listing 5.5: Verifying the production build

5.4 Docker Deployment

5.4.1 Dockerfile

The application uses a multi-stage Dockerfile:

```
# Stage 1: Build
FROM node:18-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build -- --configuration production

# Stage 2: Serve with nginx
FROM nginx:alpine
COPY --from=builder /app/dist/pnml-frontend/browser /usr/share/
    nginx/html
COPY nginx.conf /etc/nginx/nginx.conf
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

Listing 5.6: Multi-stage Dockerfile

5.4.2 Nginx Configuration

The nginx configuration handles SPA routing:

```
server {
    listen 80;
    server_name localhost;
    root /usr/share/nginx/html;
    index index.html;
```

```
6
7       # SPA fallback: serve index.html for all routes
8       location / {
9           try_files $uri $uri/ /index.html;
10      }
11
12      # Cache static assets
13      location ~* \.(js|css|png|jpg|jpeg|gif|ico|svg|woff|woff2)$
        {
14          expires 1y;
15          add_header Cache-Control "public, immutable";
16      }
17
18      # Gzip compression
19      gzip on;
20      gzip_types text/plain text/css application/json application
        /javascript;
21  }
```

Listing 5.7: nginx.conf for SPA routing

5.4.3 Building the Docker Image

```
1  # Build the image
2  docker build -t l3s-offshore-frontend:latest .
3
4  # Run the container
5  docker run -d -p 8080:80 --name offshore-frontend l3s-offshore-
       frontend:latest
6
7  # Verify it's running
8  curl http://localhost:8080
```

Listing 5.8: Building and running the Docker container

5.5 Server Deployment

5.5.1 L3S Infrastructure

The L3S research infrastructure uses a nested VPS architecture. Access requires:

1. SSH to the gateway server (`gate.l3s.uni-hannover.de`)
2. SSH to the project VPS
3. Docker commands within the VPS

5.5.2 Deployment Script

A typical deployment workflow:

```
#!/bin/bash
# deploy.sh

# 1. Build and push to container registry
docker build -t registry.l3s.de/offshore/frontend:latest .
docker push registry.l3s.de/offshore/frontend:latest

# 2. On the server: pull and restart
ssh user@server << 'EOF'
  docker pull registry.l3s.de/offshore/frontend:latest
  docker stop offshore-frontend || true
  docker rm offshore-frontend || true
  docker run -d -p 80:80 --name offshore-frontend \
    registry.l3s.de/offshore/frontend:latest
EOF
```

Listing 5.9: Production deployment script

5.5.3 Docker Cleanup

After deployment, clean up old images to save disk space:

```
# Remove unused images (not just containers)
docker image prune -a --filter "until=24h"

# Remove stopped containers
docker container prune

# Full cleanup (use with caution)
docker system prune -a
```

Listing 5.10: Docker cleanup commands

5.6 Environment Configuration

5.6.1 Production Environment

Update `src/environments/environment.prod.ts` with production URLs:

```
export const environment = {
  production: true,
  apiUrl: 'https://api.offshore.l3s.de'
};
```

Listing 5.11: Production environment configuration

5.6.2 Runtime Configuration

For dynamic configuration without rebuilding, environment variables can be injected at container runtime:

```
docker run -d -p 80:80 \
  -e API_URL=https://new-api.example.com \
  l3s-offshore-frontend:latest
```

Listing 5.12: Runtime environment injection

5.7 Troubleshooting

5.7.1 Common Issues

Blank page after deployment: Check nginx configuration and ensure `try_files` fallback is configured for SPA routing.

API connection refused: Verify the backend URL in environment configuration and check CORS settings on the backend.

Assets not loading: Ensure `base href` in `index.html` matches the deployment path.

Container exits immediately: Check logs with `docker logs <container-id>` and verify nginx configuration syntax.

5.7.2 Logging

Access container logs for debugging:

```
# Follow logs in real-time
docker logs -f offshore-frontend

# Last 100 lines
docker logs --tail 100 offshore-frontend
```

Listing 5.13: Viewing container logs

Chapter 6

Related Work

This chapter discusses existing tools and approaches for Petri net visualization and simulation, and positions the L3S-Offshore-2 Frontend within this landscape.

6.1 Desktop Petri Net Tools

6.1.1 WoPeD (Workflow Petri Net Designer)

WoPeD is a widely-used open-source tool for editing and analyzing Workflow Petri nets. It provides:

- Graphical editor for place/transition nets
- Structural analysis (e.g., place invariants)
- Token game simulation
- PNML import/export

Comparison: Unlike WoPeD, the L3S-Offshore-2 Frontend runs entirely in the browser, enabling collaboration without software installation. However, WoPeD offers more advanced analysis algorithms.

6.1.2 CPN Tools

CPN Tools is a tool for editing, simulating, and analyzing Colored Petri Nets (CPNs). It provides hierarchical modeling and state space analysis.

Comparison: CPN Tools supports more expressive Petri net types but requires desktop installation. The L3S frontend focuses on simpler P/T nets with web-based accessibility.

6.2 Web-Based Petri Net Tools

6.2.1 i-love-petrinets (Original Fork)

The L3S-Offshore-2 Frontend is based on a fork of the `i-love-petrinets` project, an Angular-based PNML viewer. The original provided:

- PNML parsing and basic visualization
- Drag-and-drop editing
- Place invariant analysis

Extensions in this project:

- Backend integration for simulation
- Animated firing sequence playback
- Offshore-specific parameter interface
- Production-ready deployment with Docker
- Significant UX improvements (zoom, pan, tab state management)

6.2.2 Cytoscape.js

Cytoscape.js is a general-purpose graph visualization library that has been extended for Petri nets. However, it lacks native PNML support and Petri net semantics.

6.3 Simulation Backends

6.3.1 gspn4py

The L3S backend uses `gspn4py`, a Python library for Generalized Stochastic Petri Nets. It provides:

- Stochastic simulation with exponential delays
- Firing sequence generation
- Performance analysis

6.3.2 pm4py

pm4py is a Python library for process mining that also supports Petri net operations. It was considered during the project's initial phase but the focus shifted to custom simulation for offshore scheduling.

6.4 Summary

The L3S-Offshore-2 Frontend fills a niche for web-based Petri net visualization with domain-specific features for offshore logistics research. While desktop tools offer more advanced analysis, the web-based approach prioritizes accessibility and collaboration.

Chapter 7

Conclusion and Future Work

This chapter summarizes the achievements of the L3S-Offshore-2 Frontend project and outlines directions for future development.

7.1 Summary

This documentation has described the development of a web-based frontend for Petri net visualization and simulation, created as part of the L3S-Offshore-2 research project.

7.1.1 Achievements

The following goals were accomplished:

1. **Functional Visualization:** The application successfully renders Petri nets from PNML files with interactive editing capabilities.
2. **Simulation Integration:** Full integration with the Flask backend enables multi-run simulations with firing sequence animation.
3. **Offshore Parameter Interface:** A dedicated tab allows researchers to configure domain-specific parameters for scheduling models.
4. **Production Deployment:** The application is containerized and deployed on L3S infrastructure, accessible via web browser.
5. **Code Quality:** Extensive JSDoc documentation and English comments ensure maintainability for future developers.

7.1.2 Technical Highlights

The most technically challenging aspects that were solved:

- **Viewport-centered zoom:** Implementing intuitive zoom behavior with ResizeObserver integration
- **Tab-state management:** Ensuring UI consistency when switching between different workflows
- **Token reset logic:** Preserving initial markings across simulation runs
- **Reactive forms integration:** Solving performance issues with Angular's form system

7.2 Limitations

The current implementation has the following limitations:

- **No data persistence:** All data is stored in-memory and lost on page reload
- **Limited to P/T nets:** Higher-level Petri net types (colored, hierarchical) are not supported
- **Single-user:** No collaborative editing or user accounts
- **Visual overlaps:** Large nets may have overlapping elements with the current layout

7.3 Future Work

The following enhancements could be pursued in future development:

7.3.1 Short-Term (High Priority)

- **Interactive Miner Integration:** Connect to pm4py's interactive miner for process discovery
- **Analysis Endpoints:** Display conformance checking results from the backend
- **Export Functions:** Enable export of simulation results as CSV/JSON

7.3.2 Medium-Term (Nice-to-Have)

- **Undo/Redo:** Implement history management for the Build tab
- **Data Persistence:** Add optional browser-based storage (IndexedDB)
- **Visual Improvements:** Reduce element overlaps with improved layout algorithms

7.3.3 Long-Term (Research Extensions)

- **Colored Petri Nets:** Extend the data model and visualization for CPNs
- **Real-Time Monitoring:** WebSocket-based live updates during long-running simulations
- **Multi-User Collaboration:** Shared editing with conflict resolution

7.4 Acknowledgements

This project was developed at the L3S Research Center, Leibniz Universität Hannover, under the supervision of Peng Qian. The author thanks the original developers of the i-love-petrinets project for providing the initial codebase.

Appendix A

Appendix: API Reference

This appendix provides a reference for the backend API endpoints used by the frontend.

A.1 Simulation Endpoints

A.1.1 Simple Simulation

Endpoint: `POST /simulation-petri-nets/simple-sim`

Description: Executes a stochastic simulation of a Petri net

Request Body: PNML content as string or file upload

Response: JSON containing firing sequence and optional detailed log

```
{
  "firing_seq": ["t1", "t3", "t2", "t1", "t4"],
  "detailed_log": [
    {"step": 0, "marking": {"p1": 1, "p2": 0}},
    {"step": 1, "marking": {"p1": 0, "p2": 1}}
  ]
}
```

Listing A.1: Example response from simple-sim

A.1.2 Example Models

Endpoint: `GET /simulation-petri-nets/example-models`

Description: Retrieves a list of available example PNML files

Response: Array of model names

Endpoint: `GET /simulation-petri-nets/example-models/<name>`

Description: Retrieves the PNML content of a specific example model

Response: PNML XML as string

A.2 Planning Endpoints

A.2.1 Default Parameters

Endpoint: `GET /dto/planning`

Description: Retrieves default parameter values for offshore scheduling

Response: JSON object with parameter definitions and defaults

A.2.2 Schedule Calculation

Endpoint: `POST /offshore-plan/calculate-schedule`

Description: Calculates an optimal schedule based on provided parameters

Request Body: JSON with scenario and workforce configuration

Response: Schedule with operation timings

A.3 Configuration Files

A.3.1 Angular Environment

`src/environments/environment.ts`:

```
export const environment = {
  production: false,
  apiUrl: 'http://localhost:9040'
};
```

Listing A.2: Development environment configuration

A.3.2 Docker Compose

`docker-compose.yml` for local development with backend:

```
version: '3.8'
services:
  frontend:
    build: .
    ports:
      - "8080:80"
    depends_on:
      - backend
  backend:
```

```
10      image: l3s-offshore-backend:latest
11      ports:
12        - "9040:9040"
```

Listing A.3: Docker Compose configuration

Bibliography